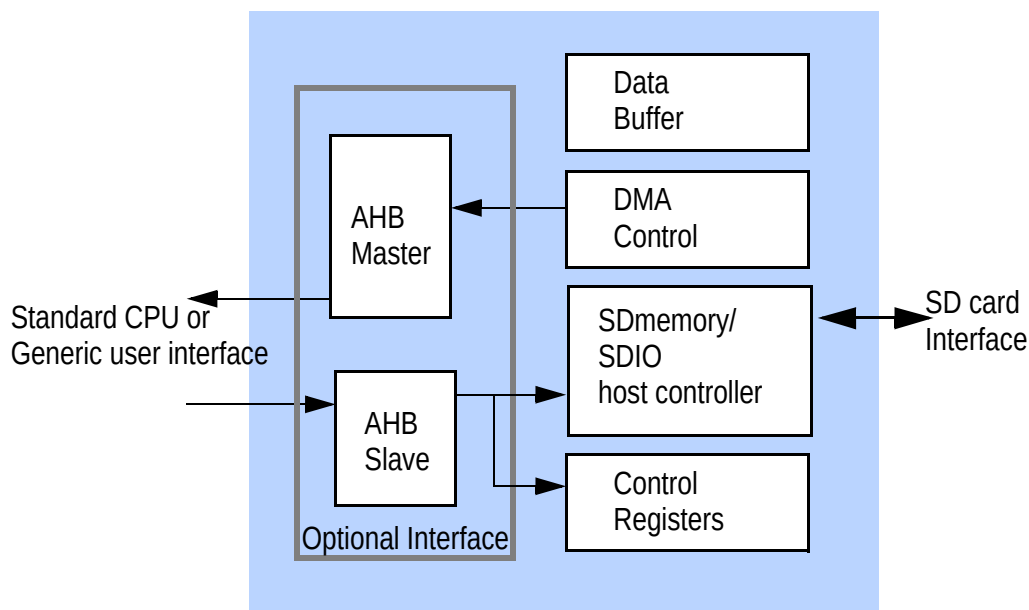




FEATURES

- Host controller for SDIO, SD memory card, and MMC interface.
- Allows host CPU to access SD and MMC devices.
- Simple user interface optimized for on-chip bus connection.
- User interface supports 32-bit and 64-bit data.
- Option to integrate with other CPU bus slaves to support direct access by various CPU's including PowerPC, MPC860, ARM, SH2/3/4, MIPS, and ARC micro-processors.
- Supports SDIO DMA operation for high speed data transfer.
- Supports SD host controller standard register set.
- Fully programmable access timing.
- Hardware support of CRC error detection and interrupt generation.
- Supports multi-function SD cards, command suspend, resume, and block transfers.
- Option to operate the user interface and card interface at different clock domains.
- Designed for ASIC and PLD implementations.
- Fully static design with edge triggered flip-flops.

BLOCK DIAGRAM



**DESCRIPTION**

The EP550 is a host controller for SD memory card, SDIO and MMC interface. The core connects the host CPU of the system to the SD card socket. External SD cards can be accessed by the host CPU through the EP550 controller core IP.

SD memory and SDIO are low cost, high speed interfaces designed for removable mass storage and IO devices. It is a very flexible architecture supporting variable clock rate and 1 to 4-bit SD data width. Data rate of up to 25Mbyte/sec (200Mbs) can be realized with SD interface. Features such as plug and play, auto-detection, error correction, write protection are standard with SD card interface.

The EP550 SD card host controller core is designed according to the SD Association's SD host controller specification. The core presents a very simple view of the SD card to the system software. All access to the SD card are made through the standard control register set. DMA, burst access, CRC error detection, interrupt, timing, etc. are supported by the controller core. Because of the standard register set, the EP550 can be used to replace other existing SD controllers with no change to system software.

There are several options for user hardware interface to the controller core. The controller supports generic user interface optimized for on-chip logic interface as well as embedded CPU interface such as AMBA AHB bus. To access the SD card, the host CPU simply issue read/write access to the control registers in the core. The controllers core handles all the SD card protocol automatically including data shifting, timing and CRC generation. The core has a built-in DMA controller so that data can be automatically transferred between the system and the SD card without CPU intervention.

With the EP550, SD card interface can be realized with very little development cost. Designer can add SD memory and SDIO interface to the system by simply adding the EP550 module without changing the rest of the system architecture.



1 Signal Descriptions

The following table listed all the external I/O signals of SD host controller. All signals with “#” or “_B” suffix are active low signals.

The following table describes the generic user interface signal for the user to access the SD controller.

Table 1: User Interface

Signal	Type	Description
U_ADDR[7:2]	I	Access address specified by the user interface.
U_ADS#	I	Address strobe. Input from the user interface to start an access.
U_BE#[3:0]	I	Byte enable. Indicates the byte(s) to be transferred.
U_BLAST#	I	Burst Last. This signal is asserted by the user interface to indicate the end of a burst transfer. For single data access, U_BLAST# is asserted as the same time as U_ADS#.
U_CS#	I	Chip select.
U_DRD[31:0]	O	Read data. This bus provides the data read from the card interface to the user interface.
U_DWR[31:0]	I	Write data from the user interface to the core.
U_RDY#	O	Data ready. This signal indicates a valid data transfer on the user interface. For read access, valid read data is presented on U_DRD at the same time as U_RDY#. For write access, U_DWR is accepted by the core when U_RDY# is asserted. For burst data transfer, U_RDY# is asserted for one cycle for each data word.

**Table 1: User Interface**

Signal	Type	Description
U_WR	I	Input from user interface to indicate read or write access. This signal is high for write and low for read.

The following table describes the SD card interface signals from the controller to the card

Table 2: SD Card Interface

Name	Type	Descriptions
SDCLK	O	Clock output to SD card
CMD	O	Burst length. Each data burst may be 4, 8 or 16 data beat long. Both cacheline wrap and linear increment are allowed.
DT[3:0]	I/O	Read data
SDCD#	I	Card Detect
SDWP#	I	Write Protect

The following table describes the generic user interface signal for the DMA controller in the SD controller to access system memory.

Table 3: User Interface

Signal	Type	Description
M_ADDR[31:0]	O	DMA address to access system memory.
M_ADS#	O	Address strobe. Output from DMA to indicate memory access.
M_BE#[3:0]	I	Byte enable. Indicates the byte(s) to be transferred.

**Table 3: User Interface**

Signal	Type	Description
M_DRD[31:0]	I	Read data. This bus provides the data read from system memory to DMA.
M_DWR[31:0]	O	Write data from the DMA to system memory.
M_ERR#	I	Error input to signal error in accessing system memory
M_RDY#	I	Data ready. This signal indicates a valid data transfer between DMA and system memory. For read access, valid read data is presented on M_DRD at the same time as M_RDY#. For write access, M_DWR is accepted by the core when M_RDY# is asserted. For burst data transfer, M_RDY# is asserted for one cycle for each data word.
M_TRANS[4:0]	O	Burst transfer size. indicates from 0 to 16 words of data to be transferred between memory and DMA.
M_WR	I	Input from user interface to indicate read or write access. This signal is high for write and low for read.

The following table lists all the miscellaneous signals:

Table 4: Miscellaneous System Signals

Name	Type	Description
CLK	I	System clock
INTR	O	Interrupt output signal to the system CPU.
RESET#	I	System reset.

**Table 4: Miscellaneous System Signals**

Name	Type	Description
SET_CLE	I	Set Current Limit Error. When the host system supply power to the card, it can be protected from cards that exceeds the current limit by stopping power supply to the card. When the electrical system of the host detects current limit error from the card, it can assert this signal for one cycle of more to set the Current Limit Error bit in the Error Interrupt Status Register (bit 7). This bit is qualified by the corresponding bits in the interrupt status enable register and interrupt signal enable register.
REG40_DT[31:0]	I	Register 40 lists the hardware and electrical capability of the host system. These are read-only data defined by the hardware to communicate with software. These register values are defined as input to the core. It should be note that the operation of the host controller core is independent of these register settings. The operation of the host controller core is based on the software writing to read/write register bits in other registers.
REG44_DT[31:0]	I	These bits are used as register40 bit 63:32. Currently these are reserved bits so the user should hardwire them to zero for the current implementation.
REG48_DT[31:0]	I	Register 48 lists the current capability of the host system. These are read-only data defined by the hardware to communicate with software. These register values are defined as input to the core. It should be note that the operation of the host controller core is independent of these register settings. The operation of the host controller core is based on the software writing to read/write register bits in other registers.

**Table 4: Miscellaneous System Signals**

Name	Type	Description
REG4C_DT[31:0]	I	These bits are used as register48 bit 63:32. Currently these are reserved bits so the user should hardwire them to zero for the current implementation.
REGFC_DT[15:0]	I	Slot Interrupt Status register. The host controller core supports only one slot so this register is ignored. These input should be hardwired to zero for the current implementation.
REGFE_DT[15:0]	I	Host Controller Version Register. User should choose a vendor specific version number and hardwire this register to the version number accordingly.
WAKEUP	O	This signal is controlled by the wakeup register. The wakeup signal should be connected to the system controller or “OR” with the “intr” output if the wakup feature is used.

1.1 Optional AHB Interface

The following table lists the optional AHB bus interface for accessing the SD controller.

Table 5: AHB Bus Slave Interface

Name	Type	Descriptions
HADDR[31:0]	I	AHB bus address received by the master
HBURST[2:0]	I	Burst length. Each data burst may be 4, 8 or 16 data beat long. Both cacheline wrap and linear increment are allowed.
HRDATA[31:0]	O	Read data
HREADYIN	I	Transfer ready. The bus slave uses this signal to qualify the address phase of a transfer.
HREADYOUT	O	Transfer ready.

**Table 5: AHB Bus Slave Interface**

Name	Type	Descriptions
HRESP[1:0]	O	Bus response.
HSEL	I	Address space select signal. This signal indicates the destination of the AHB bus access is for the SD host controller.
HSIZE[2:0]	I	Data bus size. It indicates the data size is 8-bit, 16-bit or 32-bit.
HTRANS[1:0]	I	Transfer type. It can be NONSEQUENTIAL, SEQUENTIAL, IDLE or BUSY.
HWDATA[31:0]	I	Write data.
HWRITE	I	Read/write indicator. It is high for write and low for read.

The following table lists the AHB bus signals from the SD controller DMA to the AHB bus

Table 6: AHB Bus Master Interface

Name	Type	Descriptions
M_HADDR[31:0]	O	AHB bus address driven by the master
M_HBURST[2:0]	O	Burst length. Each data burst may be 4, 8 or 16 data beat long, depending on burst internal buffer size and actual number of data to be transferred. Both cacheline wrap and linear increment are allowed on the AHB bus but the controller generates liner increment transfer only.
M_HBUSREQ	O	Bus request. Indicates the master's intention of acquiring the AHB bus to transfer data.
M_HGRANT	I	Bus grant. Driven by the system arbiter to indicate to the master that it has bus ownership.

**Table 6: AHB Bus Master Interface**

Name	Type	Descriptions
M_HLOCK	O	Bus lock. The controller does not request lock access so this signal is driven low by the controller.
M_HRDATA[31:0]	I	Read data
M_HREADY	I	Transfer ready. The signal is driven by the slave to signal data transfer.
M_HRESP[1:0]	I	Bus response. Four different responses are allowed: OKAY, ERROR, RETRY and SPLIT.
M_HSIZE[2:0]	O	Data bus size. It indicates the data size is 8-bit, 16-bit or 32-bit.
M_HTRANS[1:0]	O	Transfer type. It can be NONSEQUENTIAL, SEQUENTIAL, IDLE or BUSY.
M_HWDATA[31:0]	O	Write data.
M_HWRITE	O	Read/write indicator. It is high for write and low for read.



2. User Interface Description

This section describes the basic operation principles of the user interface. The user interface operates at the CLK clock domain. All inputs signals are synchronous with the rising edge of CLK and all output signals are driven from the rising edge of CLK.

2.1 Single read access

The operation of the user interface is based on generic microprocessor interface. User logic signals a read or write request by asserting the U_ADS# signal. Separate address, data and control signals are provided to indicate the type of transfer and the address accessed. U_WR signal is low to indicate read access. The core indicates the completion of each data transfer by asserting the U_RDY# signal. Multiple or single data can be transferred with each request.

The following timing diagram shows two read accesses, one with wait state and one with no wait state. Wait state is controlled by the core by asserting the U_RDY# signal. For read access, it is asserted when the read data becomes available on the U_DRD output. There is no limit on the number of wait cycles between U_ADS# and U_RDY#. Single access is signaled by asserting the U_BLAST# signal at the same cycle as U_ADS#.

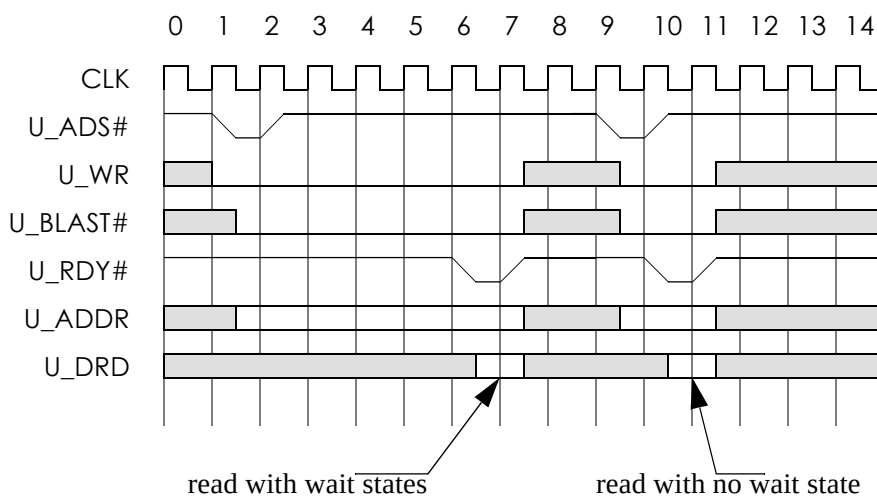


Figure 2.1 Single data read access



2.2 Burst read access

The user interface supports both single data access and burst data access. The following timing diagram shows an example of burst read access. In this example, wait state is inserted by core for the first data and last data. The core is allowed to insert any number of wait states between data. Burst access is signaled by the U_BLAST# being de-asserted at the time U_ADS# is asserted. It remains de-asserted until the transfer of the last data. Once U_BLAST# is asserted, it must remain asserted until U_RDY# is also asserted. Access terminates when U_RDY# and U_BLAST# are asserted together at the same time. There is no limit on the number of data words to be transferred with one ADS# request.

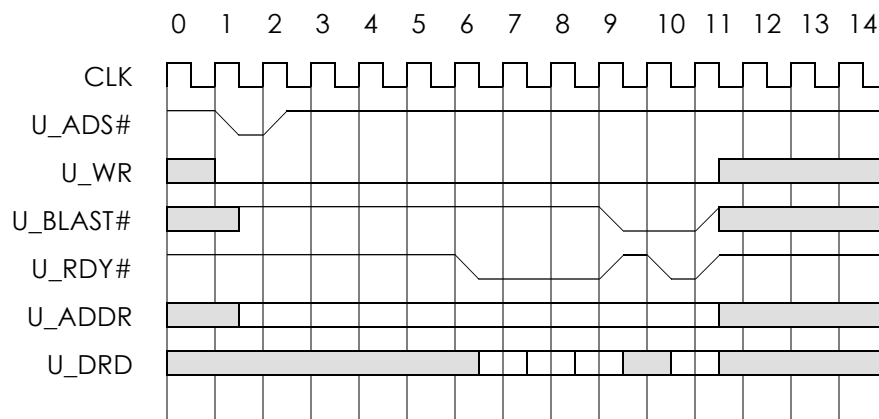


Figure 2.2 Burst data read access

2.3 Single write access

The following timing diagram shows two write accesses, one with wait state and one with no wait state. Wait state is controlled by the core by asserting the U_RDY# signal. For write access, data is driven by the external user logic at the U_DWR signals at the same cycle as it is requesting the write access (U_ADS#). It must remain valid until U_RDY# is asserted. U_RDY# signals that the core has accepted the data for write. There is no limit on the number of wait cycles between U_ADS# and U_RDY#. Single access is signaled by asserting the



U_BLAST# signal at the same cycle as U_ADS# is asserted.

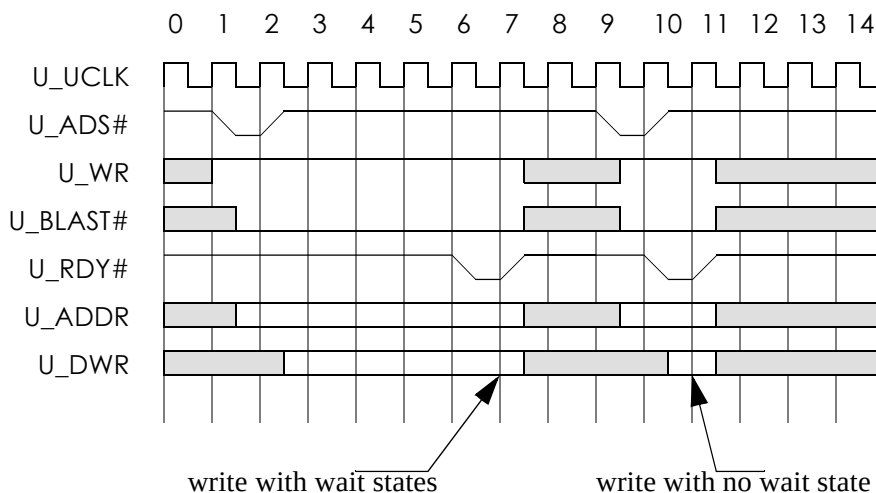


Figure 2.3 Single data write access

2.4 Burst write access

The following timing diagram shows an example of burst write access. In this example, wait state is inserted for the first data and the last data. The core is allowed to insert any number of wait states between data. Burst access is signaled by the U_BLAST# being de-asserted at the time U_ADS# is asserted. It remains de-asserted until the transfer of the last data. Once U_BLAST# is asserted, it must remain asserted until U_RDY# is also asserted. The access terminates when U_RDY# and U_BLAST# are asserted together at the same time. There is no limit



on the number of data words to be transferred with one ADS# assertion.

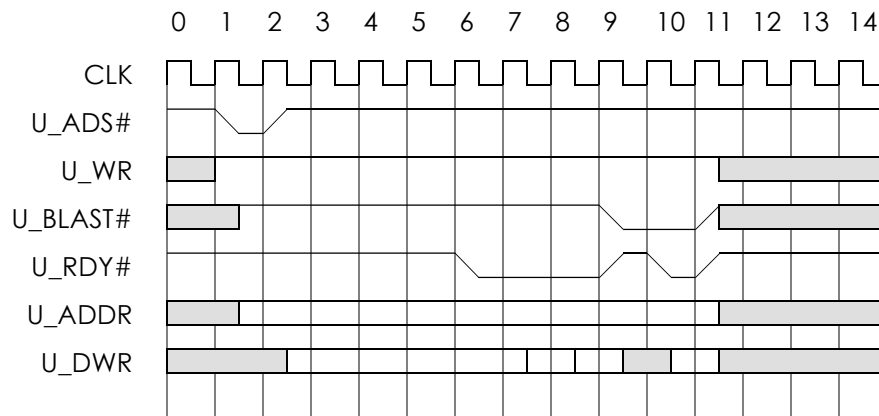


Figure 2.4 Burst data write access



3 Register Descriptions

The SD host controller implements the standard control registers as specified in the host controller specification. Access to the SD cards are handled by the system software accessing the host controllers control registers.

The following table lists all the control registers implemented by the SD host controller:

Table 7: AHB Bus Slave Interface

Address(hex)	Descriptions
000	SD DMA address (low)
002	SD DMA address (high)
004	block size
006	block count
008	argument0
00A	argument1
00C	transfer mode
00E	command
010	response0
012	response1
014	response2
016	response3
018	response4
01A	response5
01C	response6
01E	response7
020	buffer data port0
022	buffer data port1
024	present state

**Table 7: AHB Bus Slave Interface**

Address(hex)	Descriptions
026	present state
028	power control / host control
02A	wakeup control / block gap control
02C	clock control
02E	software reset / timeout control
030	normal interrupt status
032	error interrupt status
034	normal interrupt status enable
036	error interrupt status enable
038	normal interrupt signal enable
03A	error interrupt signal enable
03C	auto CMD12 error status
03E	reserved
040	capabilities
042	capabilities
044	capabilities (reserved)
046	capabilities (reserved)
048	maximum current capabilities
04A	maximum current capabilities
04C	maximum current capabilities (reserved)
04E	maximum current capabilities (reserved)
050 to 0FB	reserved
0FC	slot interrupt status
0FE	host controller version

**Table 7: AHB Bus Slave Interface**

Address(hex)	Descriptions
Others	Reserved

4 SD Control Registers

System Address Register (00h). The 2 least significant bits are hardwired to 00 for word aligned transfer.

31	0
DMA System Address[31:2]	
	0 0

Block Size Register (04h)

15	14	12	11	0
Rsvd	DMA boundary		Transfer block size	

Block Count Register (06h)

15	0
Block Count for Current Transfer	

Argument Count Register (08)

31	0
Command Argument	

Transfer Mode Register (0Ch)

15	6	5	4	3	2	1	0
Rsvd		multi/ single	Dir	rsvd	auto cmd12	block enable	DMA en



Command Register (0Eh). Both the command index (bit 13:8) and the command type controls (bit 7:0) must be set correctly for proper operation.

15-14	13-8	7-2	1-0
Rsvd	Command index	Command type & others	Response type

Response Register (10h)

31	0
Command Response 31:0	

Response Register (14h)

31	0
Command Response 63:32	

Response Register (18h)

31	0
Command Response 95:64	

Response Register (1Ch)

31	0
Command Response 127:96	

Buffer Data Port Register (20h)

31	0
Buffer Data	

Present State Register (24h). Buffer Write Enable bit (bit 10) is active after reset because space is available in the write buffer.

31	25	24	23-20	19	18	17	16
Rsvd		CMD	DAT	WP	CD	stable	insert



15	12	11	10	9	8	7-3	2	1	0
Rsvd		RD en	WR en	RTA	WTA	rsvd	DAT act	CMD-DAT inh	CMD-CMD inh

Host Control Register (28h). The LED bit (bit 0) is an internal signal to the core and is not brought out from the core in current implementation.

7-3	2	1	0
Rsvd	high speed enable	data transfer width	LED

Power Control Register (29h)

7	4	3	1	0
Rsvd		Voltage sel		SD power

*Voltage select and Power control bits are implemented but no direct outputs from the core

Block Gap Control Register (2Ah)

7	4	3	2	1	0
Rsvd		intr at block gap	read wait	continue request	stop at block gap

Wakeup Control Register (2Bh)

7	3	2	1	0
Rsvd		wakeup (removal)	wakeup (insert)	wakeup (intr)

Clock Control Register (2Ch)

15	8	7-3	2	1	0
SDCLK frequency		rsvd	SD clk enable	clock stable	clock

* test bench use SDCLK=base clock



Timeout Control Register (2Eh). The core uses the input clock, “clk”, as the base clock for the timer value. The reserved value of “1111” in data timeout counter value is mapped to 127 clocks by the core for testing purpose. This value should not be used in actual system design.

7	4	3	0
Rsvd		data timeout counter	

Software Reset Register (2Fh)

7	3	2	1	0
Rsvd		Soft reset DAT	soft reset CMD	soft reset all

Normal Interrupt Status Register (30h)

15	14	9	8	7	6	5	4	3	2	1	0
Error intr	Rsvd		card intr	card rmv	card insert	read buf ready	write buf ready	DMA intr	block gap intr	trans- fer done	cmd done

Error Interrupt Status Register (32h). There is no vendor specific interrupt in current implementation.

15	12	11	9	8	7	6	5	4	3	2	1	0
Vendor spec intr	Rsvd		auto cmd12	current limit	data end bit	data crc	data time-out	cmd index	cmd end bit	cmd crc	cmd time-out	

Normal Interrupt Enable Register (34h)

15	14	9	8	7	6	5	4	3	2	1	0
0	Rsvd		card intr	card rmv	card insert	read buf ready	write buf ready	DMA intr	block gap intr	trans- fer done	cmd done



Error Interrupt Enable Register (36h)

15	12	11	9	8	7	6	5	4	3	2	1	0
Vendor spec intr		Rsvd		auto cmd12	current limit	data end bit	data crc	data time-out	cmd index	cmd end bit	cmd crc	cmd time-out

Normal Interrupt Signal Enable Register (38h)

15	14	9	8	7	6	5	4	3	2	1	0
0	Rsvd		card intr	card rmv	card insert	read buf ready	write buf ready	DMA intr	block gap intr	transfer done	cmd done

Error Interrupt Signal Enable Register (3Ah)

15	12	11	9	8	7	6	5	4	3	2	1	0
Vendor spec intr		Rsvd		auto cmd12	current limit	data end bit	data crc	data time-out	cmd index	cmd end bit	cmd crc	cmd time-out

Auto CMD12 Error Status Register (3Ch)

15	8	7	6-5	4	3	2	1	0
rsvd		cmd err	rsvd	index error	end bit error	crc error	time-out error	not executed

Capability Register (40h)

63	0
Read-only or reserved	

Maximum Current Capability Register (48h)

63	0
Read-only or reserved	



Slot Interrupt Status Register (FCh)

15	0
Read-only or reserved	

Host Controller Version Register (FEh)

15	0
Read-only or reserved	

Eureka Technology Inc.
4962 El Camino Real
Los Altos, CA 94022, USA

Tel: 1 650 960 3800
Fax: 1 650 960 3805
email: info@eurekatech.com
<http://www.eurekatech.com>